

Instrucciones

Protocolo

CONSULTA:

- CONSULTAR <cedula> -> DATOS <cedula>|<nombre>|<apellido>|<estado> | NOT_FOUND
- LISTAR -> LISTA <n> + n líneas <cedula>|<nombre>|<apellido>|<estado>
- ESTADO <cedula> -> ESTADO <vigente|vencido|tramite> | NOT_FOUND

MODIFICACIÓN:

- CREAR <cedula> <nombre>;<apellido>;<estado> -> OK | ERROR
- ACTUALIZAR <cedula> <nombre>;<apellido>;<estado> -> OK | NOT_FOUND
- ELIMINAR <cedula> -> OK | NOT_FOUND

QUIT -> OK

Ejecutar

Servidor (Java)

```
cd server-java
javac -d out src/MainServer.java
java -cp out MainServer
```

Cliente (Python)

```
cd client-python
python3 client.py
```

Métodos implementados

Consultas (lectura)

1. ****CONSULTAR**** <cedula> → Devuelve los datos de un ciudadano.
Ejemplo: `CONSULTAR 6624465`
Respuesta: `DATOS 6624465|Elena|Ramirez|vigente`
2. ****LISTAR**** → Devuelve la lista de todos los ciudadanos registrados.
Respuesta:
LISTA 3
123|Ana|Lopez|vigente
456|Luis|Gomez|tramite
6624465|Elena|Ramirez|tramite
3. ****ESTADO**** <cedula> → Indica si el documento está vigente, vencido o en trámite.
Ejemplo: `ESTADO 6624465` → `ESTADO tramite`

Modificación (creación, actualización y eliminación)

1. ****CREAR**** ``<cedula> <nombre>;<apellido>;<estado>``

Crea un nuevo registro.

Ejemplo: ``CREAR 6624465 Elena;Ramirez;vigente`` → ``OK``

2. ****ACTUALIZAR**** ``<cedula> <nombre>;<apellido>;<estado>``

Modifica un ciudadano existente.

Ejemplo: ``ACTUALIZAR 6624465 Elena;Ramirez;tramite`` → ``OK``

3. ****ELIMINAR**** ``<cedula>``

Elimina un registro de la base de datos.

Ejemplo: ``ELIMINAR 6624465`` → ``OK``

Protocolo de comunicación (TCP)

- Los mensajes se envían como texto UTF-8 terminado en salto de línea (``\n``).
- El servidor responde con una línea de texto según el comando.
- El comando ``QUIT`` finaliza la sesión:

MainServer.java

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.concurrent.*;

public class MainServer {

    // Modelo simple
    static class Ciudadano {
        final String cedula;
        String nombre;
        String apellido;
        String estado; // vigente | vencido | tramite

        Ciudadano(String cedula, String nombre, String apellido, String
estado) {
```

```

        this.cedula = cedula;
        this.nombre = nombre;
        this.apellido = apellido;
        this.estado = estado;
    }

    String asLine() {
        return String.join("|", cedula, nombre, apellido, estado);
    }
}

// "Base de datos" en memoria segura para hilos
private static final ConcurrentHashMap<String, Ciudadano> DB = new
ConcurrentHashMap<>();

public static void main(String[] args) {
    int port = 5000;
    System.out.println("[SERVIDOR] DNIC TCP escuchando en puerto "
+ port + " ...");
    precargar(); // datos demo

    try (ServerSocket serverSocket = new ServerSocket(port)) {
        while (true) {
            Socket client = serverSocket.accept();
            System.out.println("[SERVIDOR] Cliente conectado: " +
client.getRemoteSocketAddress());
            new Thread(new ClientHandler(client)).start();
        }
    } catch (IOException e) {
        System.err.println("[SERVIDOR] Error: " + e.getMessage());
    }
}

static void precargar() {
    DB.put("123", new Ciudadano("123", "Ana", "Lopez", "vigente"));
    DB.put("456", new Ciudadano("456", "Luis", "Gomez",
"tramite"));
    DB.put("789", new Ciudadano("789", "Jose", "Ramirez",
"vigente"));
    DB.put("321", new Ciudadano("321", "Axel", "Pacheco",
"tramite"));
    DB.put("654", new Ciudadano("654", "Alan", "Paredes",
"vigente"));
}

```

```

        DB.put("987", new Ciudadano("987", "Andrea", "Ortiz",
"tramite"));
    }

    static class ClientHandler implements Runnable {
        private final Socket socket;

        ClientHandler(Socket s) {
            this.socket = s;
        }

        @Override
        public void run() {
            try {
                BufferedReader in = new BufferedReader(
                    new InputStreamReader(socket.getInputStream(),
StandardCharsets.UTF_8));
                BufferedWriter out = new BufferedWriter(
                    new OutputStreamWriter(socket.getOutputStream(),
StandardCharsets.UTF_8))
            ) {
                // Mensaje de bienvenida
                out.write("Bienvenido al servidor DNIC.\n");
                out.write("Comandos disponibles: CONSULTAR, CREAR,
ACTUALIZAR, ELIMINAR, LISTAR, QUIT\n");
                out.flush();

                String line;
                while ((line = in.readLine()) != null) {
                    System.out.println("[SERVIDOR] Comando recibido: "
+ line);

                    String resp = handle(line);
                    out.write(resp + "\n");
                    out.flush();
                    System.out.println("[SERVIDOR] Respuesta enviada: "
+ resp);

                    if ("QUIT".equalsIgnoreCase(line.trim())) break;
                }
            } catch (IOException e) {
                System.err.println("[SERVIDOR] Cliente desconectado: "
+ e.getMessage());
            } finally {
                try { socket.close(); } catch (IOException ignore) {}
            }
        }
    }

```

```

    }
}

private String handle(String line) {
    if (line == null) return "ERROR empty";
    String t = line.trim();
    if (t.isEmpty()) return "ERROR empty";

    String cmd = t.split("\\s+", 2)[0].toUpperCase();
    String rest = t.length() > cmd.length() ?
t.substring(cmd.length()).trim() : "";

    switch (cmd) {
        case "CONSULTAR": {
            if (rest.isEmpty()) return "ERROR Usage: CONSULTAR
<cedula>";

            Ciudadano c = DB.get(rest);
            return (c == null) ? "NOT_FOUND" : "DATOS " +
c.asLine();
        }

        case "LISTAR": {
            List<Ciudadano> list = new
ArrayList<>(DB.values());
            StringBuilder sb = new StringBuilder();
            sb.append("LISTA
").append(list.size()).append("\n");
            for (Ciudadano c : list)
sb.append(c.asLine()).append("\n");
            return sb.toString().trim();
        }

        case "ESTADO": {
            if (rest.isEmpty()) return "ERROR Usage: ESTADO
<cedula>";

            Ciudadano c = DB.get(rest);
            return (c == null) ? "NOT_FOUND" : "ESTADO " +
c.estado;
        }

        case "CREAR": {
            // CREAR <cedula> <nombre>;<apellido>;<estado>
            String[] p = rest.split("\\s+", 2);

```

```

        if (p.length < 2) return "ERROR Usage: CREAR
<cedula> <nombre>;<apellido>;<estado>";
        String ced = p[0];
        String[] campos = p[1].split(";", 3);
        if (campos.length < 3) return "ERROR Usage: CREAR
<cedula> <nombre>;<apellido>;<estado>";
        if (DB.containsKey(ced)) return "ERROR Ya existe";
        if (!campos[2].matches("vigente|vencido|tramite"))
            return "ERROR Estado inválido. Use:
vigente|vencido|tramite";
        DB.put(ced, new Ciudadano(ced, campos[0],
campos[1], campos[2]));
        return "OK";
    }

    case "ACTUALIZAR": {
        // ACTUALIZAR <cedula> <nombre>;<apellido>;<estado>
        String[] p = rest.split("\\s+", 2);
        if (p.length < 2) return "ERROR Usage: ACTUALIZAR
<cedula> <nombre>;<apellido>;<estado>";
        String ced = p[0];
        String[] campos = p[1].split(";", 3);
        if (campos.length < 3) return "ERROR Usage:
ACTUALIZAR <cedula> <nombre>;<apellido>;<estado>";
        Ciudadano c = DB.get(ced);
        if (c == null) return "NOT_FOUND";
        if (!campos[2].matches("vigente|vencido|tramite"))
            return "ERROR Estado inválido. Use:
vigente|vencido|tramite";
        c.nombre = campos[0];
        c.apellido = campos[1];
        c.estado = campos[2];
        return "OK";
    }

    case "ELIMINAR": {
        if (rest.isEmpty()) return "ERROR Usage: ELIMINAR
<cedula>";
        return (DB.remove(rest) == null) ? "NOT_FOUND" :
"OK";
    }

    case "QUIT":

```

```

        return "OK";

        default:
            return "ERROR Unknown command";
    }
}
}

```

MainServer\$ClientHandler.class

// Source code is decompiled from a .class file using FernFlower decompiler (from IntelliJ IDEA).

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Iterator;

class MainServer$ClientHandler implements Runnable {
    private final Socket socket;

    MainServer$ClientHandler(Socket var1) {
        this.socket = var1;
    }

    public void run() {
        try {
            BufferedReader var1 = new BufferedReader(new
InputStreamReader(this.socket.getInputStream(),
StandardCharsets.UTF_8));

            try {
                BufferedWriter var2 = new BufferedWriter(new
OutputStreamWriter(this.socket.getOutputStream(),
StandardCharsets.UTF_8));

                String var3;
                try {

```

```

        while((var3 = var1.readLine()) != null) {
            String var4 = this.handle(var3);
            var2.write(var4);
            var2.write("\n");
            var2.flush();
            if ("QUIT".equalsIgnoreCase(var3.trim())) {
                break;
            }
        }
    } catch (Throwable var19) {
        try {
            var2.close();
        } catch (Throwable var18) {
            var19.addSuppressed(var18);
        }

        throw var19;
    }

    var2.close();
} catch (Throwable var20) {
    try {
        var1.close();
    } catch (Throwable var17) {
        var20.addSuppressed(var17);
    }

    throw var20;
}

var1.close();
} catch (IOException var21) {
    System.err.println("[SERVIDOR] Cliente desconectado: " +
var21.getMessage());
} finally {
    try {
        this.socket.close();
    } catch (IOException var16) {
    }

}

}
}

```



```

private String handle(String var1) {
    if (var1 == null) {
        return "ERROR empty";
    } else {
        String var2 = var1.trim();
        if (var2.isEmpty()) {
            return "ERROR empty";
        } else {
            String var3 = var2.split("\\s+", 2)[0].toUpperCase();
            String var4 = var2.length() > var3.length() ?
var2.substring(var3.length()).trim() : "";
            String[] var7;
            String var8;
            String[] var9;
            MainServer$Ciudadano var10;
            MainServer$Ciudadano var11;
            switch (var3) {
                case "CONSULTAR":
                    if (var4.isEmpty()) {
                        return "ERROR Usage: CONSULTAR <cedula>";
                    }

                    var11 =
(MainServer$Ciudadano)MainServer.DB.get(var4);
                    return var11 == null ? "NOT_FOUND" : "DATOS " +
var11.asLine();
                case "LISTAR":
                    ArrayList var12 = new
ArrayList(MainServer.DB.values());
                    StringBuilder var13 = new StringBuilder();
                    var13.append("LISTA
").append(var12.size()).append("\n");
                    Iterator var14 = var12.iterator();

                    while (var14.hasNext()) {
                        var10 = (MainServer$Ciudadano)var14.next();
                        var13.append(var10.asLine()).append("\n");
                    }

                    return var13.toString().trim();
                case "ESTADO":
                    if (var4.isEmpty()) {

```

```

        return "ERROR Usage: ESTADO <cedula>";
    }

    var11 =
(MainServer$Ciudadano)MainServer.DB.get(var4);
    return var11 == null ? "NOT_FOUND" : "ESTADO " +
var11.estado;

    case "CREAR":
        var7 = var4.split("\\s+", 2);
        if (var7.length < 2) {
            return "ERROR Usage: CREAR <cedula>
<nombre>;<apellido>;<estado>";
        } else {
            var8 = var7[0];
            var9 = var7[1].split(";", 3);
            if (var9.length < 3) {
                return "ERROR Usage: CREAR <cedula>
<nombre>;<apellido>;<estado>";
            } else {
                if (MainServer.DB.containsKey(var8)) {
                    return "ERROR Ya existe";
                }

                MainServer.DB.put(var8, new
MainServer$Ciudadano(var8, var9[0], var9[1], var9[2]));
                return "OK";
            }
        }

    case "ACTUALIZAR":
        var7 = var4.split("\\s+", 2);
        if (var7.length < 2) {
            return "ERROR Usage: ACTUALIZAR <cedula>
<nombre>;<apellido>;<estado>";
        } else {
            var8 = var7[0];
            var9 = var7[1].split(";", 3);
            if (var9.length < 3) {
                return "ERROR Usage: ACTUALIZAR <cedula>
<nombre>;<apellido>;<estado>";
            } else {
                var10 =
(MainServer$Ciudadano)MainServer.DB.get(var8);
                if (var10 == null) {

```



```

    String asLine() {
        return String.join("|", this.cedula, this.nombre, this.apellido,
this.estado);
    }
}

```

MainServer.class

```

// Source code is decompiled from a .class file using FernFlower
decompiler (from IntelliJ IDEA).
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.concurrent.ConcurrentHashMap;

public class MainServer {
    private static final ConcurrentHashMap<String, MainServer$Ciudadano>
DB = new ConcurrentHashMap();

    public MainServer() {
    }

    public static void main(String[] var0) {
        short var1 = 5000;
        System.out.println("[SERVIDOR] DNIC TCP escuchando en puerto " +
var1 + " ...");
        precargar();

        try {
            ServerSocket var2 = new ServerSocket(var1);

            try {
                while(true) {
                    Socket var3 = var2.accept();
                    System.out.println("[SERVIDOR] Cliente conectado: " +
String.valueOf(var3.getRemoteSocketAddress()));
                    (new Thread(new
MainServer$ClientHandler(var3))).start();
                }
            } catch (Throwable var6) {
                try {
                    var2.close();

```

```

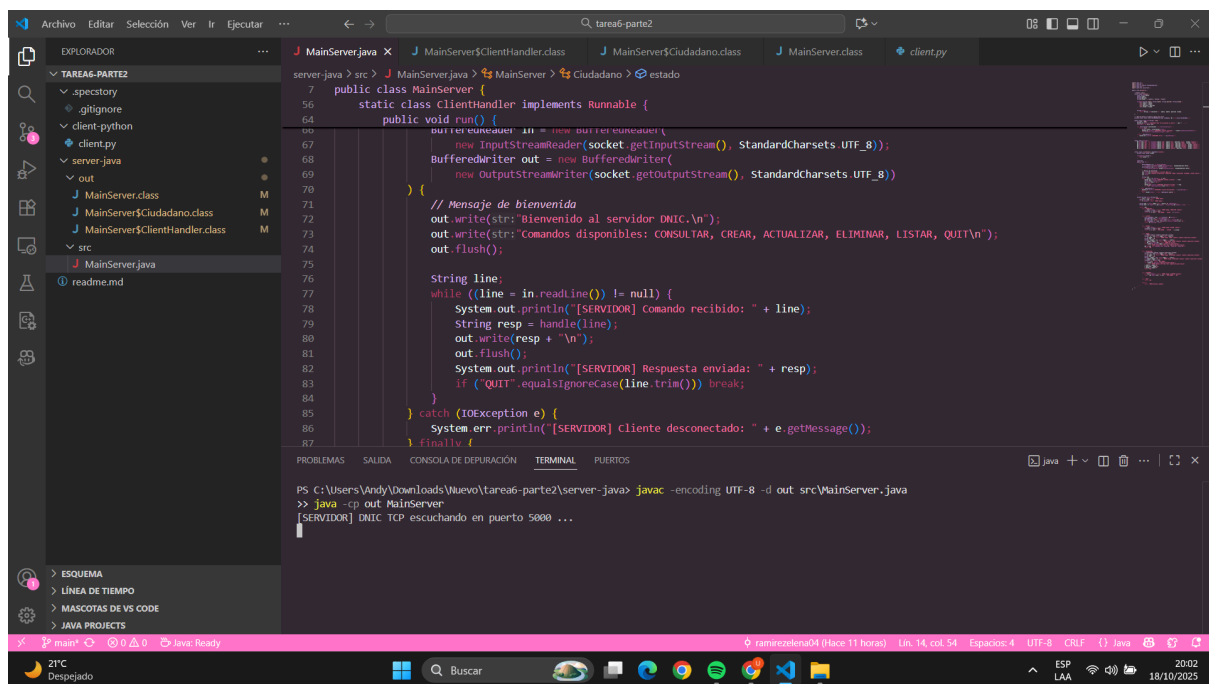
    } catch (Throwable var5) {
        var6.addSuppressed(var5);
    }

    throw var6;
}
} catch (IOException var7) {
    System.err.println("[SERVIDOR] Error: " + var7.getMessage());
}
}

static void precargar() {
    DB.put("123", new MainServer$Ciudadano("123", "Ana", "Lopez",
"vigente"));
    DB.put("456", new MainServer$Ciudadano("456", "Luis", "Gomez",
"tramite"));
}
}

```

Inicio



Menú

```
[CLIENTE] Conectado a 127.0.0.1:5000  
Bienvenido al servidor DNIC.  
Comandos disponibles: CONSULTAR, CREAR, ACTUALIZAR, ELIMINAR, LISTAR, QUIT
```

```
--- MENÚ ---
```

```
1. CREAR ciudadano  
2. ACTUALIZAR ciudadano  
3. CONSULTAR ciudadano  
4. ESTADO ciudadano  
5. LISTAR todos  
6. ELIMINAR ciudadano  
7. SALIR  
Seleccione opción: 5
```